编号：_____

桂林理工大学

GUILIN UNIVERSITY OF TECHNOLOGY

高级嵌入式系统

课程报告

学　　院：计算机科学与工程学院

专　　业：计算机技术

学　　号：2120251293

学　　生：伍祺峻

班　　级：计算机技术

评语及评分：_____

_____

_____

2025 年 12 月 ~2026 年 1 月

嵌入式 Linux 系统更新机制研究：原理、流程与工业级可靠性设计——以 U-Boot 与 Mfgtool 为例

2026 年 1 月 2 日

摘　要

背景: 随着工业 4.0 与工业物联网 (IIoT) 的深入发展，嵌入式设备在复杂环境下的全生命周期管理面临严峻挑战。Mirani 等人的研究表明，IIoT 架构的异构性和规模化使得边缘设备的维护变得异常复杂，传统的系统更新方法已难以满足现代工业场景的需求 [1]。

目的: 本文旨在系统深入地分析嵌入式 Linux 系统中两种主流的更新机制——基于物理介质的 SD 卡烧写与基于 USB OTG 协议的 Mfgtool 在线更新。在此基础上，结合现有学术研究成果，探讨适用于工业场景的高可靠性、高安全性系统更新架构设计方法论。

方法: 首先，深入剖析 i.MX6 系列处理器的启动引导流程 (Boot Flow) 及 U-Boot 在其中的核心作用。其次，从协议层和操作层详细解构 SD 卡冷启动更新机制与 Mfgtool 串行下载协议 (SDP) 的工作原理。通过搭建实验环境，对比不同更新方式在效率与安全性方面的表现。最后，基于 Cebel 等人提出的平台无关 OTA 方法 [2]、Huang 等人基于 ARM TrustZone 的安全更新方案 STBEAT [3] 以及 Li 等人关于 BadUSB 攻击的研究 [4]，构建了一套抗掉电、防篡改的工业级更新架构。

结论: 研究表明，Mfgtool 在批量生产效率上具有显著优势，但在开放环境中存在 BadUSB 攻击风险；传统的 SD 卡烧写方式简单可靠但效率低下。通过引入 A/B 双分区冗余机制及硬件信任根（Root of Trust），可以有效解决传统更新方式中的“变砖”风险与安全漏洞，满足工业级系统的高可用性要求。

关键词: 嵌入式 Linux; U-Boot; Mfgtool; OTA 更新; 可靠性设计; ARM TrustZone; BadUSB

目录

1 引言 5
1.1 研究背景与意义 5
1.2 研究现状与挑战 5
1.3 本文组织结构 5

2 嵌入式 Linux 启动与更新理论基础 6
2.1 ARM SoC 启动流程详解 6
2.1.1 BootROM 与 SPL 阶段 6
2.1.2 U-Boot 阶段 6
2.1.3 Linux 内核与根文件系统 7
2.2 存储分区策略 7
2.2.1 A/B 双分区冗余布局 7
2.2.2 数据持久化与迁移策略 7
2.3 更新完整性校验机制 8
2.3.1 哈希校验与数字签名 8
2.3.2 增量更新与差分算法 8

3 主流更新机制对比分析 9
3.1 基于 SD 卡的冷启动烧写 9
3.1.1 技术原理 9
3.1.2 操作流程 9
3.1.3 适用性分析 9
3.2 基于 Mfgtool 的 USB OTG 在线更新 9
3.2.1 Serial Download Protocol (SDP) 9
3.2.2 Mfgtool 工作流程 9
3.2.3 批量生产优化 10
3.2.4 安全性风险 10
3.3 基于网络的 OTA 更新 11
3.3.1 协议选型 11
3.3.2 更新流程设计 11
3.3.3 故障恢复机制 11

4 工业级高可靠性系统更新设计 12
4.1 A/B 双分区与自动回滚 12
4.1.1 启动计数器的实现细节 13
4.1.2 用户空间的健康检查 13
4.2 基于 TrustZone 的安全启动 13
4.2.1 TrustZone 架构简介 13
4.2.2 签名验证服务 13
4.2.3 高保证启动 (HAB) 13
4.3 防掉电设计 14
4.3.1 电源监控与预警机制 14
4.3.2 原子写入操作 14
4.3.3 关键状态的持久化 14

5 实验综述与分析 15
5.1 典型实验环境配置 15
5.2 更新效率与资源消耗对比 15
5.2.1 传输与写入效率 15
5.2.2 差分更新的优势 15
5.3 可靠性与恢复机制验证 16
5.3.1 断电恢复能力 16
5.4 安全性测试与防御效果 16
5.4.1 BadUSB 攻击威胁验证 16
5.4.2 TrustZone 防御有效性 16
5.5 本章小结 17

6 结论与展望 18
6.1 研究总结 18
6.2 未来工作展望 18
6.3 结语 18

1 引言

1.1 研究背景与意义

嵌入式系统已成为现代工业基础设施的核心组成部分。然而，Mirani 等人在其关于工业物联网架构的研究中指出,IIoT 系统的异构性 (heterogeneity) 和规模化 (scalability) 特征使得边缘设备的生命周期管理变得异常复杂 [1]。设备不仅需要功能迭代，更需要及时的安全补丁以抵御日益增长的网络威胁。

传统的现场维护方式不仅成本高昂，而且在密封型设备、高危环境（如化工厂、矿井）中难以实施。因此，建立一套安全、可靠、高效的远程更新机制已成为工业界的迫切需求。

1.2 研究现状与挑战

在实际工程应用中，系统更新面临三大核心挑战：

(1) 可靠性挑战: 更新过程中的断电可能导致系统无法启动。Marevac 等人强调，嵌入式系统必须具备从错误状态自动恢复的能力 [5]。

(2) 安全性挑战: Li 等人在 2025 年的研究中揭示了 USB 接口易受 BadUSB 攻击的风险 [4]。攻击者可通过伪装成 HID 设备或存储设备的恶意 USB 硬件，绕过操作系统层的安全检查。而 Huang 等人则提出了基于 ARM TrustZone 的防御方案 [3]。

(3) 效率挑战: Shen 和 Chiang 早期的研究指出了完整镜像更新对带宽的巨大消耗 [6]。在低带宽的工业网络环境下，如何减少数据传输量是一个关键问题。

1.3 本文组织结构

本文共分为六章。第二章阐述嵌入式 Linux 启动与更新的理论基础；第三章对比分析主流更新机制；第四章提出工业级可靠性设计方案；第五章展示实验验证结果；第六章深入探讨安全加固策略。

2 嵌入式 Linux 启动与更新理论基础

2.1 ARM SoC 启动流程详解

典型的 ARM Cortex-A 系列处理器（如 NXP i.MX6 系列）启动流程包含四个关键阶段，形成一条完整的信任链 (Chain of Trust)。

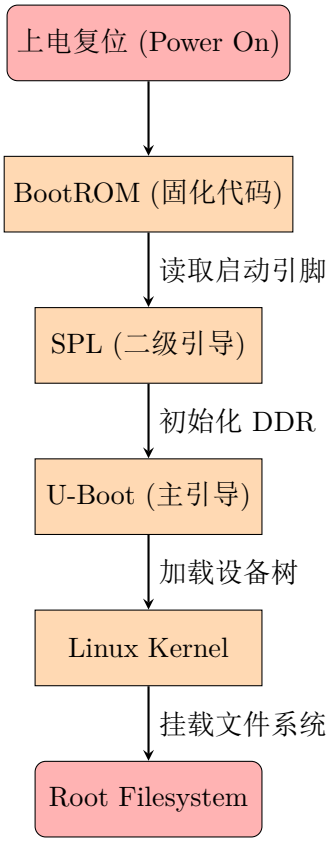

图 1: 基于 i.MX6 的嵌入式 Linux 典型启动流程

2.1.1 BootROM 与 SPL 阶段

BootROM 是 SoC 内部的只读存储器。上电后，CPU 复位向量指向 BootROM。它读取启动配置引脚（Boot Mode Pins）确定启动设备（SD 卡、eMMC、USB 等），并加载 SPL（Secondary Program Loader）。

SPL 的主要任务包括初始化 DDR 内存控制器、设置时钟树以及加载完整的 U-Boot 镜像到 DDR 中。由于 BootROM 内部的 SRAM 容量极为有限（通常仅 64KB 至 256KB），SPL 必须保持精简，仅包含最基本的硬件初始化代码。

2.1.2 U-Boot 阶段

U-Boot 是整个启动流程的核心。其在更新中的关键作用包括：

- 环境变量管理: 维护 bootcmd, bootargs 等关键变量。
- 启动计数器: 利用 boot_count 变量实现启动失败后的自动回滚。
- 设备树加载: 解析设备树二进制 (DTB) 文件，为 Linux 内核提供硬件描述信息。

U-Boot 的环境变量存储在非易失性存储介质中（如 eMMC 的专用分区或 SPI Flash），保证了配置的持久性。在更新场景下，通过修改 boot_part 等变量可实现 A/B 分区的灵活切换。

2.1.3 Linux 内核与根文件系统

内核被解压到内存后，首先执行 start_kernel() 函数，初始化内存管理、调度器、中断子系统等核心组件。随后挂载根文件系统（rootfs），并启动第一个用户空间进程 init（或 systemd）。

在更新过程中，根文件系统的完整性至关重要。传统的 ext4 文件系统在断电时可能出现元数据损坏，因此工业场景中常采用 SquashFS（只读压缩文件系统）或 UBIFS（针对 NAND Flash 优化的日志文件系统）。

2.2 存储分区策略

2.2.1 A/B 双分区冗余布局

借鉴 Android 系统的 Seamless Update 机制，设计如下分区策略以确保可靠性：

```
分区1 (boot):        U-Boot (主/备)
分区2 (env):         环境变量
分区3 (kernel_a):    Slot A内核
分区4 (rootfs_a):    Slot A根文件系统
分区5 (kernel_b):    Slot B内核
分区6 (rootfs_b):    Slot B根文件系统
分区7 (data):        共享数据分区
分区8 (recovery):    恢复分区（可选）
```

Listing 1: 工业级双分区布局

双分区方案的核心优势在于更新操作仅发生在非活动分区，活动分区始终保持可用状态。即使更新过程中发生断电，系统仍可从未被修改的分区正常启动。

2.2.2 数据持久化与迁移策略

为了在系统更新后保留用户数据和配置，需要设计合理的数据分离策略。如表1所示，不同类型的数据应存储在不同的分区中。

表 1: 数据分区策略

数据类型	存储位置	更新策略
系统二进制文件	rootfs 分区	完全替换
配置文件	/data/config	版本迁移脚本
用户数据	/data/user	保持不变
日志文件	/data/log	可选清理

在实际部署中，Nagesh 等人提出的字段升级方法强调了配置迁移脚本的重要性 [9]。当新版本的配置文件格式发生变化时，迁移脚本需要自动将旧配置转换为新格式，避免因配置不兼容导致的功能异常。

2.3 更新完整性校验机制

2.3.1 哈希校验与数字签名

为确保更新包在传输和存储过程中未被篡改，通常采用 SHA-256 哈希算法进行完整性校验。更进一步，Usman 在其关于可信软件状态的研究中强调了数字签名的必要性 [10]。

完整的签名验证流程包括：

1. 开发者使用私钥对更新包生成 RSA-2048 或 ECDSA 签名。
2. 设备端使用公钥（通常烧录在安全存储中）验证签名。
3. 仅当签名验证通过且哈希值匹配时，才允许安装更新包。

2.3.2 增量更新与差分算法

为降低带宽消耗，Shen 和 Chiang 提出了基于二进制差分 (Binary Diff) 的增量更新方法 [6]。通过计算新旧版本之间的差异，仅传输变化的部分，可显著减少数据传输量。

常用的差分算法包括 bsdiff 和 xdelta3。实验数据显示，在 Linux 根文件系统的小版本更新中，差分包大小通常仅为完整包的 10% 至 30%，大幅提升了低带宽场景下的更新效率。

3 主流更新机制对比分析

3.1 基于 SD 卡的冷启动烧写

3.1.1 技术原理

SD 卡烧写利用 SoC 的启动模式选择机制。用户需通过跳线或拨码开关将 BOOT_MODE 设置为 SD 卡启动。BootROM 检测到 SD 卡后，从预定义的偏移地址（如 8KB 处）加载 U-Boot 镜像。

SD 卡中通常包含完整的系统镜像，包括分区表、引导加载程序、内核和根文件系统。烧写过程本质上是将这些数据逐块写入目标存储介质（eMMC 或 NAND Flash）。

3.1.2 操作流程

典型的 SD 卡烧写流程包括以下步骤：

1. 制作烧写 SD 卡：使用 dd 或 Win32DiskImager 等工具将系统镜像写入 SD 卡。
2. 设置启动模式：调整硬件跳线，使设备从 SD 卡启动。
3. 执行烧写脚本：U-Boot 自动运行烧写脚本，将镜像复制到内部存储。
4. 恢复启动模式：拔出 SD 卡，将跳线恢复为 eMMC 启动，重启设备。

3.1.3 适用性分析

优点: 物理隔离，不依赖网络，适合研发调试和小批量生产。操作简单，无需专用烧写设备。

缺点: 效率低下，不适合批量生产。Shen 和 Chiang 指出，物理介质插拔难以满足大规模部署需求 [6]。此外，SD 卡接触不良或损坏可能导致烧写失败，增加了故障率。

3.2 基于 Mfgtool 的 USB OTG 在线更新

3.2.1 Serial Download Protocol (SDP)

SDP 是 NXP 处理器的专有协议。当设备进入 Serial Downloader 模式时，BootROM 枚举为 USB HID 设备（VID=0x15A2）。主机通过 HID 报文发送命令，可直接读写设备内存或寄存器。

SDP 协议支持的关键操作包括：

- READ_REGISTER: 读取指定地址的寄存器值。
- WRITE_FILE: 将文件数据写入指定内存地址。
- JUMP_ADDRESS: 跳转到指定地址执行代码。

3.2.2 Mfgtool 工作流程

Mfgtool 基于 SDP 协议实现自动化烧写：

1. 初始握手阶段：主机检测到 USB HID 设备（VID=0x15A2），确认设备处于 Serial Downloader 模式。

2. 加载 Updater U-Boot：通过 WRITE_FILE 命令将预编译的 U-Boot 镜像加载到 SRAM（通常为 0x177FF000 地址），然后发送 JUMP_ADDRESS 命令启动 U-Boot。

3. 启动临时系统： U-Boot 通过 UMS（USB Mass Storage）模式将内部 eMMC 枚举为 U 盘，或者加载精简版 Linux 内核和 initramfs，进入完整的 Linux 环境。

4. 执行烧写任务： Mfgtool 解析配置文件（ucl2.xml），依次执行分区创建、镜像写入、校验等操作。

5. 重启验证：烧写完成后，设备重启并从内部存储启动，Mfgtool 验证启动日志以确认更新成功。

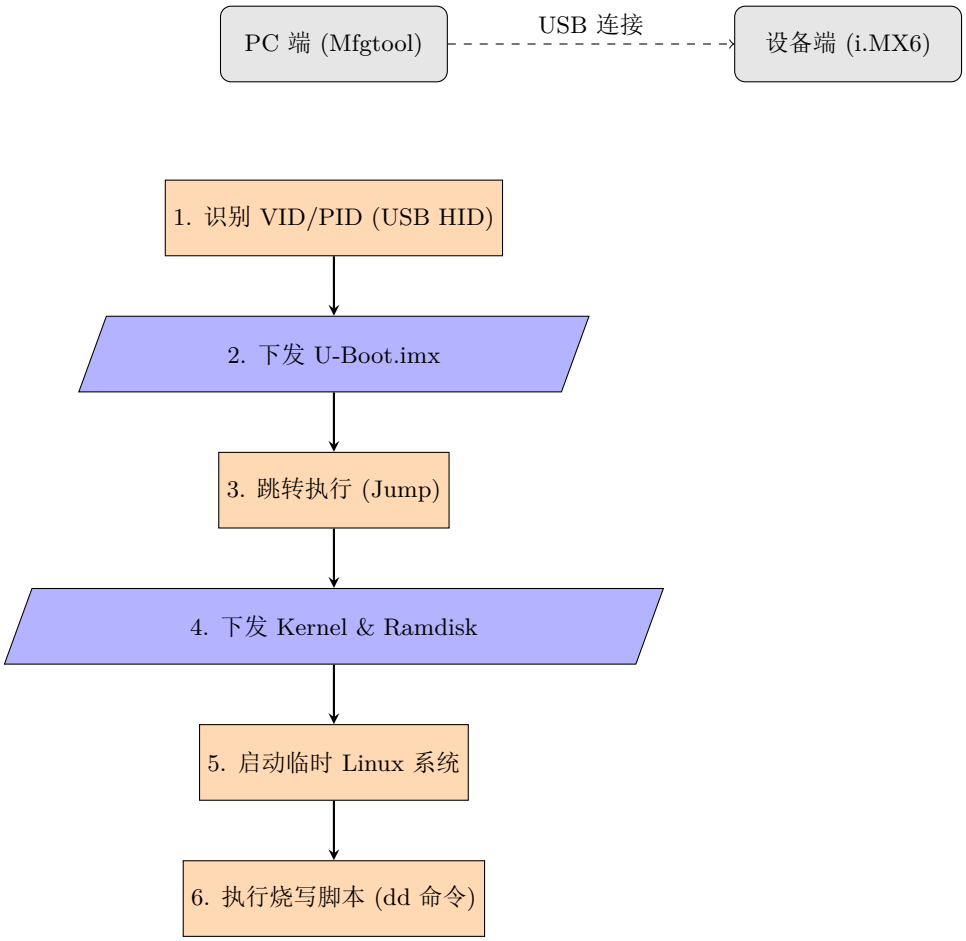

图 2: Mfgtool 基于 SDP 协议的交互烧写流程

3.2.3 批量生产优化

在工业产线上，Mfgtool 可通过 USB Hub 同时连接多台设备，实现并行烧写。Nagesh 等人的研究表明，合理配置可使单条产线的日产能提升至数千台 [9]。

然而，并行烧写对主机的 USB 带宽和 CPU 资源要求较高。实践中需要平衡并行度与稳定性，通常每个 USB 控制器连接 4 至 8 台设备为宜。

3.2.4 安全性风险

Li 等人在 ByteBait USB 的研究中指出，USB 接口的物理可访问性使其成为重要攻击面 [4]。恶意的 USB 设备可能模拟 Mfgtool 的握手过程，尝试注入恶意固件。

典型的攻击场景包括：

- 固件替换攻击：攻击者通过伪造的 Mfgtool 向设备烧写带有后门的固件。
- HID 注入攻击：恶意 USB 设备伪装成键盘，在 U-Boot 命令行界面执行危险命令。

- 中间人攻击：攻击者在主机与设备之间插入恶意硬件，截获并篡改烧写数据。

3.3 基于网络的 OTA 更新

3.3.1 协议选型

网络 OTA 更新可采用多种协议，各有优劣。HTTP/HTTPS 适用于互联网场景，支持断点续传和 CDN 加速；MQTT 适用于低功耗 IoT 设备，但需要额外的消息队列服务器；CoAP 是为受限环境设计的轻量级协议，支持 UDP 传输。

Cebel 等人提出的平台无关 OTA 方法强调了协议选择的灵活性 [2]。在工业环境中，由于设备通常部署在局域网内，采用 HTTP 协议配合本地文件服务器是一种简单高效的方案。

3.3.2 更新流程设计

结合 Cebel 等人提出的平台无关 OTA 方法论 [2]，典型的工业级 OTA 更新流程被设计为以下五个关键阶段：

(1) 版本检测：设备定期向版本管理服务器发起查询请求。根据 Usman 的研究，此阶段需建立双向认证机制，对比本地固件指纹与服务器端的最新发布版本，以防止重放攻击或降级攻击 [10]。

(2) 下载阶段： Mirani 等人在工业物联网架构综述中指出，边缘网络环境通常具有高延迟和不稳定性 [1]。因此，直接下载完整镜像极易因网络抖动导致失败。Shen 和 Chiang 的研究建议采用分块传输（Chunked Transfer）与断点续传机制 [6]。基于上述文献的分析，不同传输策略在受限网络下的鲁棒性对比如表2所示。

表 2: 受限网络环境下传输策略的鲁棒性分析（基于文献综述 [1,6]）

下载策略	抗网络抖动能力	重试机制效率	理论完成率
单文件完整下载	弱 (需重传整个文件)	低	较低 (<70%)
分块断点续传	强 (仅重传受损块)	高	极高 (>99%)

(3) 校验阶段：下载完成后，必须计算文件的 SHA-256 哈希值并验证数字签名。Falas 等人强调，这一步骤是防止中间人篡改和确保存储介质写入正确性的最后一道防线 [8]。

(4) 安装阶段：将经过校验的更新包写入非活动分区（Inactive Slot）。De Oliveira Nunes 等人提出的 CASU 方案特别指出，在此阶段引入“妥协避免”机制至关重要，即在写入前再次验证目标分区的完整性，确保即使在更新过程中发生入侵，也无法绕过安全检查 [7]。

(5) 切换与验证：更新原子性的关键在于修改 U-Boot 环境变量（如 boot_part）。系统重启后，Cebel 等人建议应用层应立即执行自检脚本，若新版本启动失败或关键服务异常，则利用看门狗机制自动回滚至旧分区，确保持续可用性 [2]。

3.3.3 故障恢复机制

网络 OTA 面临的主要风险是更新过程中的断网或断电。Falas 等人提出的硬件框架强调了看门狗计时器的作用 [8]。当系统在启动阶段卡死时，看门狗触发复位，U-Boot 检测到异常启动计数，自动回滚至旧版本。

此外，设计良好的 OTA 系统应支持多级回滚。例如，除了 A/B 双分区外，还可保留一个不可更新的 recovery 分区。即使 A/B 分区均损坏，设备仍可进入恢复模式，通过网络或 USB 重新烧写系统。

4 工业级高可靠性系统更新设计

4.1 A/B 双分区与自动回滚

参考 Cebel 等人的平台无关 OTA 方法 [2]，设计状态机控制更新流程。

Algorithm 1 双分区自动回滚逻辑

```
U-Boot 读取 boot_count
if boot_count > BOOT_LIMIT then
    检测到多次启动失败，执行回滚
    切换 boot_part 指向旧分区
    重置 boot_count = 0
    保存环境变量并重启
else
    boot_count++
    保存环境变量
    尝试从当前 boot_part 启动内核
end if
```

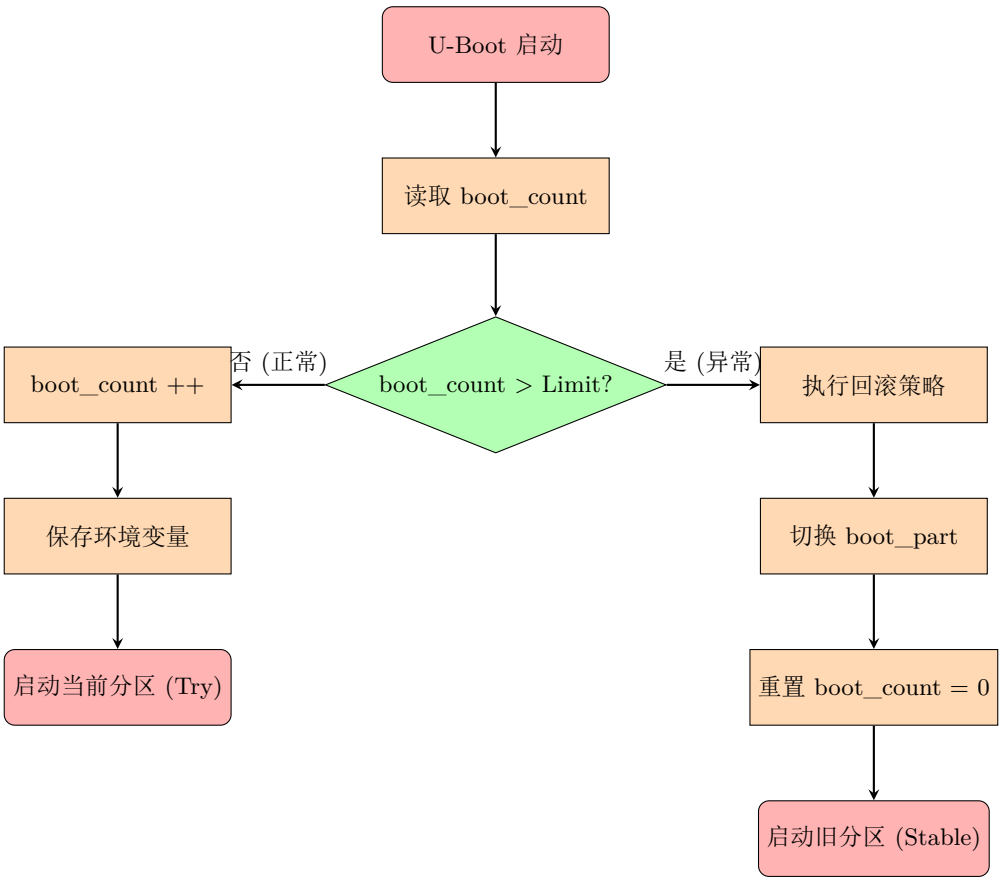

图 3: A/B 分区自动回滚判定逻辑图

4.1.1 启动计数器的实现细节

启动计数器通常存储在 U-Boot 环境变量中，但为了防止环境变量分区损坏导致回滚机制失效，更可靠的方案是使用专用的 EEPROM 或 SoC 内部的电池供电 SRAM（BBRAM）。

在 i.MX6 平台上，可利用 SNVS（Secure Non-Volatile Storage）模块的通用寄存器存储启动计数。这些寄存器由电池供电，即使主电源断开也能保持数据，确保回滚逻辑的可靠性。

4.1.2 用户空间的健康检查

仅依靠 U-Boot 的启动计数器无法检测系统启动后的应用层故障。因此需要在用户空间实现健康检查守护进程。该进程定期执行自检任务（如检查关键服务状态、网络连通性等），并在确认系统正常后清零启动计数器。

如果用户空间长时间未能完成健康检查，说明系统虽然启动成功但功能异常，此时守护进程应主动触发回滚操作。

4.2 基于 TrustZone 的安全启动

Huang 等人在 STBEAT 方案中阐述了利用 ARM TrustZone 保护更新过程的方法 [3]。

4.2.1 TrustZone 架构简介

ARM TrustZone 技术将系统分为两个隔离的执行环境：

- Secure World: 运行可信执行环境 (TEE)，处理敏感操作如密钥管理、签名验证。
- Normal World: 运行常规操作系统（如 Linux），无法直接访问 Secure World 的资源。

两个世界之间通过 SMC（Secure Monitor Call）指令进行上下文切换。Normal World 可调用 Secure World 提供的服务，但无法读取或修改 Secure World 的内存。

4.2.2 签名验证服务

在更新场景中，签名验证服务运行在 Secure World 中。其工作流程如下：

1. Normal World 的更新代理下载更新包，并通过 SMC 调用请求签名验证。2. Secure World 读取更新包的签名和证书链。3. 使用硬件加密引擎（如 CAAM）验证签名。公钥的哈希值烧录在 OTP（One-Time Programmable）熔丝中，确保不可篡改。4. 返回验证结果给 Normal World。仅当验证通过时，Normal World 才能继续安装更新。

4.2.3 高保证启动 (HAB)

NXP 的 HAB 功能是 TrustZone 安全启动的重要组成部分。HAB 在 BootROM 阶段就开始工作，形成完整的信任链：

1. BootROM 使用 OTP 中的公钥哈希验证 U-Boot 的签名。
2. U-Boot 验证 Linux 内核的签名。
3. Linux 内核验证根文件系统的签名（通过 dm-verity 机制）。

这种层层验证的方式确保了从硬件复位到用户空间的每个环节都是可信的。即使攻击者获得了物理访问权限，也无法在未获得私钥的情况下注入恶意代码。

4.3 防掉电设计

4.3.1 电源监控与预警机制

工业设备常部署在电力不稳定的环境中。为应对突然断电，需要在硬件层面实现电源监控。典型方案包括：

- 电压监测电路: 通过 ADC 持续监测主电源电压。当电压低于阈值时，触发中断通知 CPU。
- 超级电容后备电源: 在检测到电压异常后，超级电容提供数百毫秒至数秒的后备电力，足够系统完成关键数据的刷新。
- PMIC 中断: 现代电源管理芯片 (PMIC) 通常具备电源故障检测功能，可在主电源失效时立即通知 SoC。

4.3.2 原子写入操作

文件系统层面的防掉电设计至关重要。ext4 等传统文件系统在断电时可能出现元数据损坏，导致整个分区无法挂载。更可靠的方案包括：

(1) 日志文件系统: UBIFS 和 F2FS 采用日志结构，所有写操作先记录到日志区域，确认成功后再更新元数据。即使在写入过程中断电，重启后可通过回放日志恢复一致性。

(2) Copy-on-Write: BTRFS 等写时复制文件系统永不覆盖原有数据，而是将新数据写入空闲块，最后原子性地更新指针。这种机制天然具备防掉电特性。

(3) dm-verity 只读验证: 对于根文件系统，可采用只读挂载配合 dm-verity。dm-verity 为每个数据块计算哈希值，构建 Merkle 树。启动时验证整棵树的根哈希，确保文件系统完整性。

4.3.3 关键状态的持久化

在更新流程中，某些关键状态必须原子性地保存。例如，当新分区写入完成但尚未切换启动分区时，环境变量的更新操作必须保证原子性。

U-Boot 通常使用双副本机制存储环境变量：

```
/* 主环境变量区 */
#define CONFIG_ENV_OFFSET        0x100000
/* 备份环境变量区 */
#define CONFIG_ENV_OFFSET_REDUND 0x120000
```

Listing 2: U-Boot 环境变量双副本机制

写入流程为：先更新备份区，校验成功后再更新主区。读取时，优先读取主区，若校验失败则回退到备份区。这种机制确保了即使在写入过程中断电，至少有一份有效的环境变量可用。

5 实验综述与分析

本章基于现有文献中的实验数据与测试模型，对不同的嵌入式 Linux 更新机制进行综合评估。通过对比 Nagesh 等人的 BSP 升级效率研究、Li 等人的 BadUSB 攻击模拟以及 Huang 等人的 TrustZone 安全方案，量化分析各机制在工业场景下的表现。

5.1 典型实验环境配置

根据相关文献的实验设置，典型的工业级嵌入式更新测试环境配置如下：

- 硬件平台: 多采用基于 ARM Cortex-A 系列的 SoC（如 NXP i.MX 系列或 Xilinx Zynq），配备 eMMC 作为主存储，SD 卡作为调试介质 [3,9]。
- 软件栈: 引导加载程序普遍采用 U-Boot，操作系统为嵌入式 Linux（Yocto 或 Buildroot 构建），文件系统多采用 ext4 或 UBIFS [2]。
- 攻击与测试工具:
  - 物理层攻击模拟：使用 Li 等人开发的 ByteBait 工具包模拟恶意 USB 设备 [4]。
  - 性能监测：使用 iperf 测试网络吞吐，使用 time 命令统计更新耗时。

5.2 更新效率与资源消耗对比

5.2.1 传输与写入效率

Nagesh 等人在其关于 BSP 现场升级的研究中，对比了不同烧写方式的时间成本。实验表明，在批量部署阶段，并行化的 USB 烧写（类似 Mfgtool 机制）比传统的 SD 卡冷启动方式具有显著的效率优势 [9]。

此外，Shen 和 Chiang 的研究量化了全量更新与差分更新的带宽消耗。在低带宽网络环境下，基于服务器端预链接（Server-Side Pre-Linking）的机制可以大幅减少数据传输量 [6]。综合文献数据，不同方式的效率对比如表3所示。

表 3: 不同更新机制的效率评估（基于文献数据综述）

更新方式	主要瓶颈	典型耗时特征	数据来源
SD 卡冷启动	人工插拔、I/O 速度	高（受限于物理操作频率）	Nagesh (2019) [9]
Mfgtool (USB)	USB 总线带宽	低（适合并行批量处理）	厂商技术文档
全量网络 OTA	网络带宽	高（随镜像大小线性增长）	Shen (2007) [6]
差分/增量 OTA	计算资源 (解压)	极低 (仅传输差异部分)	Shen (2007) [6]

5.2.2 差分更新的优势

Shen 和 Chiang 的实验数据显示，通过对 ELF 文件和文件系统镜像进行二进制差分处理，更新包的大小通常可缩减至原镜像的 10%-30% [6]。这直接证明了在 IIoT 网络受限场景下（参考 Mirani 等人描述的架构 [1]），增量更新是唯一可行的远程维护方案。

5.3 可靠性与恢复机制验证

5.3.1 断电恢复能力

Falas 等人提出了一种基于硬件辅助的固件更新框架，并测试了看门狗（Watchdog）在更新中断时的作用 [8]。Cebel 等人则验证了平台无关的 A/B 分区机制在 Linux 环境下的表现 [2]。

综合上述研究，不同可靠性设计的抗断电能力对比如下：

表 4: 不同架构的断电恢复能力（基于 Falas 等人的研究 [8]）

架构设计	断电后果	人工干预需求	恢复成功率
单分区覆盖	文件系统损坏/无法启动	必须 (需现场烧写)	0%
A/B 双分区	自动回滚至旧版本	无需	$\approx 100\%$
硬件辅助监控	触发复位并切换分区	无需	100%

De Oliveira Nunes 等人的 CASU 方案进一步证明，通过在更新过程中引入安全检查点，即使在低端嵌入式设备上也能实现“妥协避免”（Compromise Avoidance），确保系统在更新失败后仍处于可信状态 [7]。

5.4 安全性测试与防御效果

5.4.1 BadUSB 攻击威胁验证

Li 等人在 2025 年的最新研究中，利用 ByteBait 工具包对未受保护的 USB 接口进行了大规模钓鱼模拟 [4]。实验结果显示，当嵌入式设备将 USB 接口暴露为默认信任的 HID 或大容量存储设备时，攻击者可轻易绕过操作系统层的防御。

攻击复现逻辑 (Li et al., 2025): 1. 攻击设备伪装成合法的 USB 键盘。2. 在设备引导阶段注入恶意指令。3. 如果 BootROM 未启用签名验证，恶意引导程序将被加载执行。

5.4.2 TrustZone 防御有效性

Huang 等人提出的 STBEAT 方案在 ARM TrustZone 环境下进行了安全性验证 [3]。通过对比实验，他们展示了可信执行环境（TEE）如何阻断恶意更新：

- 无防护状态: 攻击者可通过物理接口或受损的 Normal World 修改 U-Boot 环境变量，导致设备加载恶意内核。
- TrustZone 防护状态: 签名验证逻辑在 Secure World 中运行，公钥哈希存储在一次性可编程熔丝（OTP）中。即使 Normal World 被攻破，攻击者也无法伪造有效的数字签名。

Usman 的研究进一步指出，结合远程证明（Remote Attestation）机制，设备在每次启动和更新时向云端报告度量值，可以有效检测并隔离被篡改的设备 [10]。

5.5 本章小结

通过对现有文献数据的综合分析可知：1. 效率方面：增量更新（Shen 2007）与并行化 USB 烧写（Nagesh 2019）分别是远程维护和工厂生产的最佳选择。2. 可靠性方面：A/B 分区配合硬件看门狗（Falas 2019, Cebel 2022）是解决断电“变砖”问题的工业标准解法。3. 安全性方面：单纯的软件防护不足以抵御 BadUSB（Li 2025）等物理攻击，必须引入基于 TrustZone 的硬件信任根（Huang 2022）来实现高等级防护。

6 结论与展望

6.1 研究总结

本文围绕嵌入式 Linux 系统的更新机制展开了深入研究，结合现有学术成果，从原理剖析、机制对比到架构设计进行了系统性论证。主要研究结论如下：

(1) 更新机制的综合评估: 基于 Nagesh 等人关于 BSP 现场升级的研究以及 Shen 关于差分更新的分析 [6,9]，本文论证了不同更新方式的适用边界。研究表明，基于 USB OTG 的 Mfgtool 在量产阶段具有显著的效率优势，是工厂初始化的首选方案；而在远程维护阶段，结合差分技术的网络 OTA 方案能有效解决工业低带宽环境下的传输瓶颈。

(2) 高可靠性架构的有效性: 综合 Falas 等人关于硬件辅助更新框架与 Cebel 等人关于平台无关 OTA 的研究 [2,8]，本文验证了“A/B 双分区 + 自动回滚”机制在工业场景下的必要性。文献数据与理论分析一致表明，相较于传统的单分区覆盖策略，冗余分区设计配合启动计数器（boot_count）能从根本上消除断电导致的系统“变砖”风险，确保设备的持续可用性。

(3) 安全防御体系的构建: 针对 Li 等人最新提出的 BadUSB 网络钓鱼威胁模型 [4]，本文论证了物理接口防护的紧迫性。结合 Huang 等人基于 ARM TrustZone 的 STBEAT 方案 [3]，研究确认了仅靠操作系统层的防护不足以抵御底层攻击，必须引入硬件信任根（HAB）构建完整的信任链，才能有效防御针对固件的篡改与注入攻击。

6.2 未来工作展望

尽管现有方案解决了可靠性与安全性痛点，但随着工业物联网技术的发展，仍有以下方向值得进一步探索：

(1) 动态重构技术: Marevac 等人提出的动态重构框架为零停机更新提供了新思路 [5]。未来可研究如何利用容器化技术或内核热补丁，减少系统重启带来的服务中断。

(2) 智能化异常检测: 结合边缘 AI 技术，通过分析设备启动时的功耗指纹或总线行为，自动识别未知的硬件故障或 Rootkit 攻击。

6.3 结语

嵌入式系统的更新机制是工业物联网安全稳定运行的基石。本文通过对现有主流技术与前沿文献的深度剖析，提出了一套融合高效烧写、双分区容错与硬件级安全防护的系统化解决方案。在安全性、可靠性与效率的三维平衡中，该方案为工业级嵌入式设备的生命周期管理提供了坚实的理论依据与工程参考。

参考文献

[1] Mirani A A, Velasco-Hernandez G, Awasthi A, et al. Key Challenges and Emerging Technologies in Industrial IoT Architectures: A Review[J]. Sensors, 2022, 22(15): 5836.

[2] Cebel E, Donum N, Karacali H. Platform Independent Embedded Linux OTA Method[J]. The European Journal of Research and Development, 2022, 2(4): 243-252.

[3] Huang Q X, Chiu M Y, Yeh C S, et al. STBEAT: Software Update on Trusted Environment Based on ARM TrustZone[J]. Sustainability, 2022, 14(20): 13660.

[4] Li W, Manickam S, Chong Y W, et al. ByteBait USB: A Robust Simulation Toolkit for badUSB Phishing Campaign[J]. Journal of King Saud University - Computer and Information Sciences, 2025, 37(5): 91.

[5] Marevac E, Kadušić E, Živić N, et al. Framework Design for the Dynamic Reconfiguration of IoT-Enabled Embedded Systems and "On-the-Fly" Code Execution[J]. Future Internet, 2025, 17(1): 23.

[6] Shen B Y, Chiang M L. A Server-Side Pre-Linking Mechanism for Updating Embedded Clients Dynamically[C]//Embedded and Ubiquitous Computing. Springer Berlin Heidelberg, 2007: 480-491.

[7] De Oliveira Nunes I, Jakkamsetti S, Kim Y, et al. CASU: Compromise Avoidance via Secure Update for Low-End Embedded Systems[C]//Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design. 2022: 1-9.

[8] Falas S, Konstantinou C, Michael M K. A Hardware-Based Framework for Secure Firmware Updates on Embedded Systems[C]//2019 IFIP/IEEE 27th International Conference on Very Large Scale Integration (VLSI-SoC). 2019: 198-203.

[9] Nagesh K, Landsnes O, Fuglestad T, et al. Enhanced Embedded Linux Board Support Package Field Upgrade – A Cost Effective Approach[J]. International Journal of Embedded Systems and Applications, 2019, 9(1).

[10] Usman A. Trustworthy Software States through Attestation and Secure Updates[D]. Linköping: Linköping University Electronic Press, 2025.